Sistemas Embebidos

Laboratorio 1: MCU, sensores y actuadores

Ejercicio 3

14 Abril, 2025

1. Ejercicio 3

En este ejercicio se implementó un benchmark en el ESP32-S3 para medir el desempeño de un kernel simple compuesto por operaciones aritméticas enteras y de punto flotante. El programa replica el pseudocódigo solicitado en el enunciado, utilizando las variables `var_1 = 233` y `var_2 = 128`, y ejecutando dentro de un ciclo las siguientes operaciones:

- `result_0 = var_1 + var_2`
- `result_1 = var_1 + 10`
- `result_2 = var_1% var_2`
- `result_3 = var_1 * var_2`
- `result_4 = var_1 / var_2`

Para cada prueba se midieron:

- el tiempo total de ejecución mediante `esp_timer_get_time()`,
- la cantidad de ciclos de reloj mediante `esp_cpu_get_cycle_count()`,
- y un CPI estimado a partir de una estimación estática de instrucciones por iteración.

El experimento se realizó variando tanto el número de iteraciones X como la frecuencia del MCU. La idea fue analizar cómo evoluciona el CPI al aumentar la carga de trabajo, identificar qué operación resulta más costosa dentro del kernel y estudiar cómo cambia el comportamiento del sistema al modificar la frecuencia del procesador.

A continuación se presentan los resultados obtenidos a través del monitor serial para las tres frecuencias evaluadas.

Resultados del benchmark completo

Cuadro 1: Resultados del benchmark del kernel para distintas frecuencias y distintos valores de X.

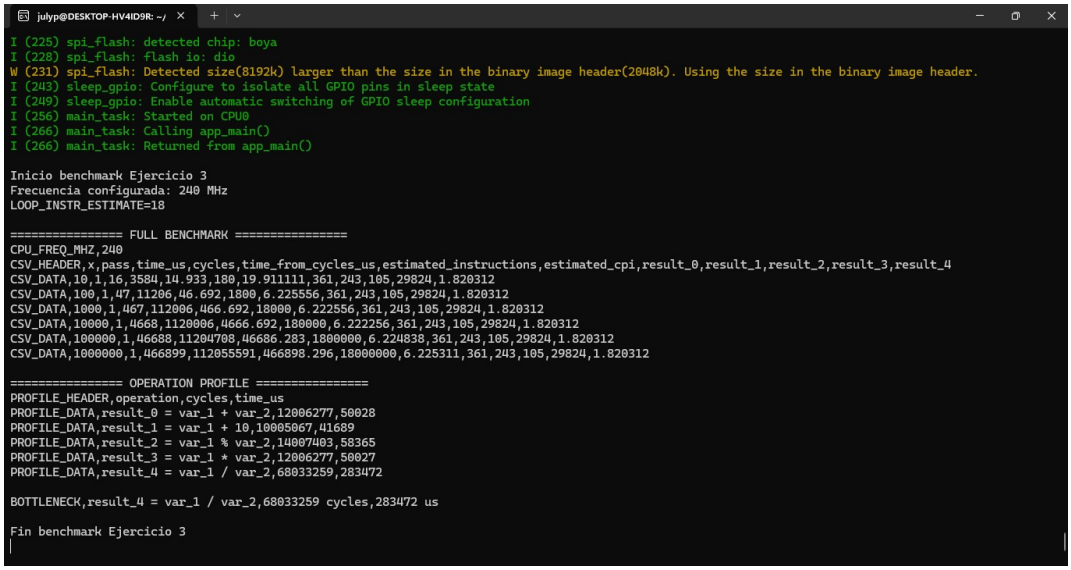
Frecuencia	X	Pass	Tiempo [μs]	Ciclos	Tiempo por ciclos [μs]	Instr. estimadas	CPI estimado
240 MHz	10	1	16	3584	14.933	180	19.9111
240 MHz	100	1	47	11206	46.692	1800	6.2256
240 MHz	1000	1	467	112006	466.692	18000	6.2226
240 MHz	10000	1	4668	1120006	4666.692	180000	6.2223
240 MHz	100000	1	46688	11204708	46686.283	1800000	6.2248
240 MHz	1000000	1	466899	112055591	466898.296	18000000	6.2253
160 MHz	10	1	18	2816	17.600	180	15.6444
160 MHz	100	1	71	11206	70.038	1800	6.2256
160 MHz	1000	1	701	112006	700.038	18000	6.2226
160 MHz	10000	1	7008	1121160	7007.250	180000	6.2287
160 MHz	100000	1	70049	11207760	70048.500	1800000	6.2265
160 MHz	1000000	1	700485	112077499	700484.369	18000000	6.2265
80 MHz	10	1	28	2040	25.500	180	11.3333
80 MHz	100	1	142	11206	140.075	1800	6.2256
80 MHz	1000	1	1401	112006	1400.075	18000	6.2226
80 MHz	10000	1	14016	1121093	14013.662	180000	6.2283
80 MHz	100000	1	140195	11215480	140193.500	1800000	6.2308
80 MHz	1000000	1	1401947	112155662	1401945.775	18000000	6.2309

Resultados del profiling por operación

Cuadro 2: Resultados del profiling de operaciones aritméticas para un millón de iteraciones.

Frecuencia	Operación	Ciclos	Tiempo [μs]
240 MHz	`result_0 = var_1 + var_2`	12006277	50028
240 MHz	`result_1 = var_1 + 10`	10005067	41689
240 MHz	`result_2 = var_1 % var_2`	14007403	58365
240 MHz	`result_3 = var_1 * var_2`	12006277	50027
240 MHz	`result_4 = var_1 / var_2`	68033259	283472
160 MHz	`result_0 = var_1 + var_2`	12008032	75052
160 MHz	`result_1 = var_1 + 10`	10006897	62545
160 MHz	`result_2 = var_1 % var_2`	14010168	87564
160 MHz	`result_3 = var_1 * var_2`	12009131	75058
160 MHz	`result_4 = var_1 / var_2`	68047326	425297
80 MHz	`result_0 = var_1 + var_2`	12016225	150206
80 MHz	`result_1 = var_1 + 10`	10013009	125165
80 MHz	`result_2 = var_1 % var_2`	14019359	175243
80 MHz	`result_3 = var_1 * var_2`	12016197	150204
80 MHz	`result_4 = var_1 / var_2`	68091496	851146

Figura 1: Evidencia de la ejecución del benchmark en el monitor serial.



```
julyp@DESKTOP-HV4ID9R: ~/
I (225) spi_flash: detected chip: boya
I (228) spi_flash: flash io: dio
W (231) spi_flash: Detected size(8192k) larger than the size in the binary image header(2048k). Using the size in the binary image header.
I (243) sleep_gpio: Configure to isolate all GPIO pins in sleep state
I (249) sleep_gpio: Enable automatic switching of GPIO sleep configuration
I (256) main_task: Started on CPU0
I (266) main_task: Calling app_main()
I (266) main_task: Returned from app_main()

Inicio benchmark Ejercicio 3
Frecuencia configurada: 240 MHz
LOOP_INSTR_ESTIMATE=18

================ FULL BENCHMARK ================
CPU_FREQ_MHZ,240
CSV_HEADER,x,pass,time_us,cycles,time_from_cycles_us,estimated_instructions,estimated_cpi,result_0,result_1,result_2,result_3,result_4
CSV_DATA,10,1,16,3584,14.933,180,19.911111,361,243,105,29824,1.820312
CSV_DATA,100,1,47,11206,46.692,1800,6.225556,361,243,105,29824,1.820312
CSV_DATA,1000,1,467,112006,466.692,18000,6.222556,361,243,105,29824,1.820312
CSV_DATA,10000,1,4668,1120006,4666.692,180000,6.222256,361,243,105,29824,1.820312
CSV_DATA,100000,1,46688,11204708,46686.283,1800000,6.224838,361,243,105,29824,1.820312
CSV_DATA,1000000,1,466899,112055591,466898.296,18000000,6.225311,361,243,105,29824,1.820312

================ OPERATION PROFILE ================
PROFILE_HEADER,operation,cycles,time_us
PROFILE_DATA,result_0 = var_1 + var_2,12006277,50028
PROFILE_DATA,result_1 = var_1 + 10,10005067,41689
PROFILE_DATA,result_2 = var_1 % var_2,14007403,58365
PROFILE_DATA,result_3 = var_1 * var_2,12006277,50027
PROFILE_DATA,result_4 = var_1 / var_2,68033259,283472

BOTTLENECK,result_4 = var_1 / var_2,68033259 cycles,283472 us

Fin benchmark Ejercicio 3
```

Figura 1: Ejecución del benchmark en el monitor serial.

1.1. Análisis de Rendimiento

1.1.1. Evolución del CPI al aumentar X

Al mirar cómo cambia el CPI estimado en función de X, se ve que para valores pequeños, especialmente en $X = 10$, aparecen valores bastante altos. Por ejemplo, a 240 MHz se obtuvo un CPI estimado de 19.91, mientras que a 160 MHz y 80 MHz se obtuvieron 15.64 y 11.33, respectivamente. Esto no significa que el kernel realmente se comporte así en régimen normal, sino que en esos casos el overhead fijo de la medición pesa mucho más. Ahí influyen la inicialización de temporizadores, la lectura del contador de ciclos, la entrada y salida de la función y el propio control del loop.

A medida que X aumenta, ese efecto fijo se reparte entre muchas más iteraciones, y el CPI empieza a estabilizarse. De hecho, para $X \geq 100$, el CPI converge a un valor muy parecido en los tres casos, quedando alrededor de 6.22–6.23. Esto muestra que, una vez que la carga de trabajo es suficientemente grande, el comportamiento del programa se vuelve mucho más estable y representativo.

También se observa que el CPI prácticamente no cambia al modificar la frecuencia. Esto tiene sentido, porque el CPI mide ciclos por instrucción y no tiempo absoluto. En otras palabras, al cambiar la frecuencia cambia cuánto dura cada ciclo, pero no cambia demasiado la cantidad de ciclos que necesita el kernel para hacer el mismo trabajo.

De todas formas, hay que considerar que en este experimento el número de instrucciones no fue medido directamente con un contador hardware real, sino estimado a partir de una constante fija por iteración. Por eso, el valor obtenido corresponde a un **CPI estimado**, que sirve muy bien para estudiar tendencias y comparar comportamiento, aunque no debe tomarse como una medición exacta en valor absoluto.

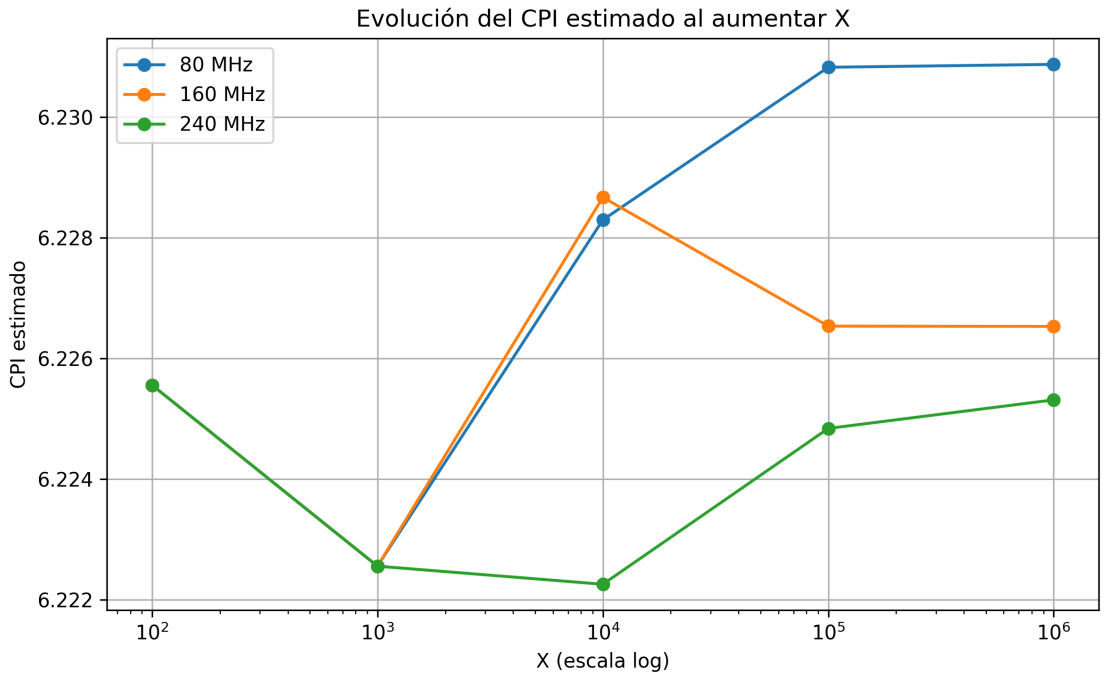


Figura 2: Evolución del CPI estimado en función de X para 80, 160 y 240 MHz.

1.1.2. Profiling del programa y determinación del bottleneck

El profiling por operación deja bastante claro que la instrucción más costosa del kernel es la división en punto flotante:

```
result_4 = var_1 / var_2
```

En las tres frecuencias evaluadas, esta operación consume cerca de 68 millones de ciclos para un millón de iteraciones, mientras que el resto de las operaciones se mueve aproximadamente entre 10 y 14 millones de ciclos. En particular, a 240 MHz la suma con constante utiliza cerca de 10 millones de ciclos, la suma entre variables y la multiplicación alrededor de 12 millones, el módulo cerca de 14 millones, y la división llega a unos 68 millones.

Esto muestra que la división requiere varias veces más ciclos que las demás operaciones del kernel. Por lo mismo, se puede identificar sin mucha duda como el **bottleneck** del programa, ya que es la instrucción que más pesa dentro del costo total de ejecución.

Este resultado también es razonable desde el punto de vista de arquitectura. Las operaciones de suma y multiplicación suelen estar muy optimizadas en hardware, mientras que una división, y más aún una división en punto flotante, tiende a ser bastante más costosa.

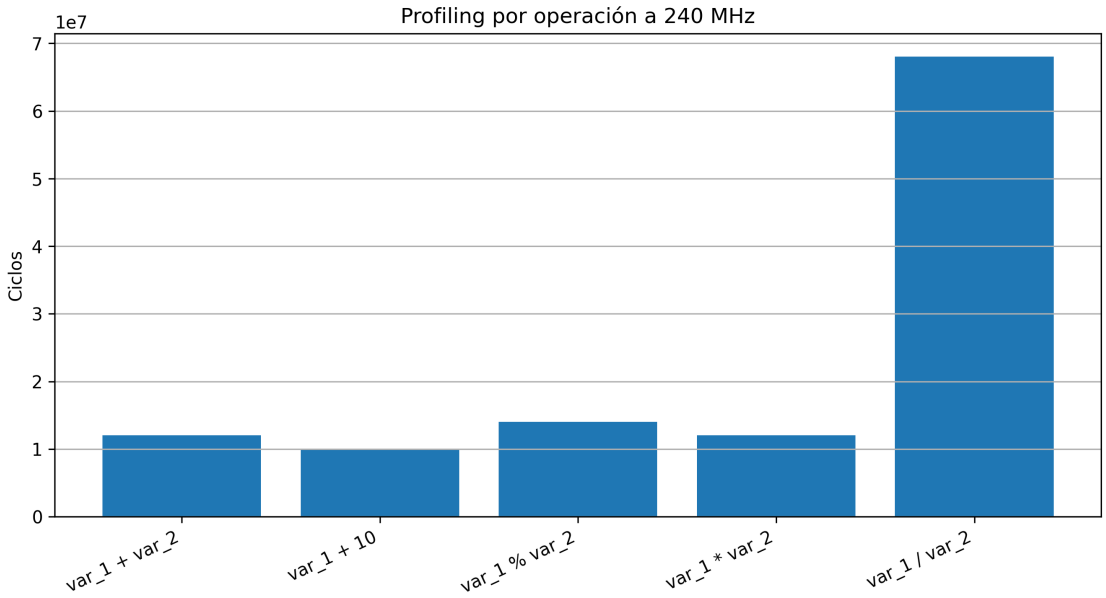


Figura 3: Número de ciclos por operación a 240 MHz.

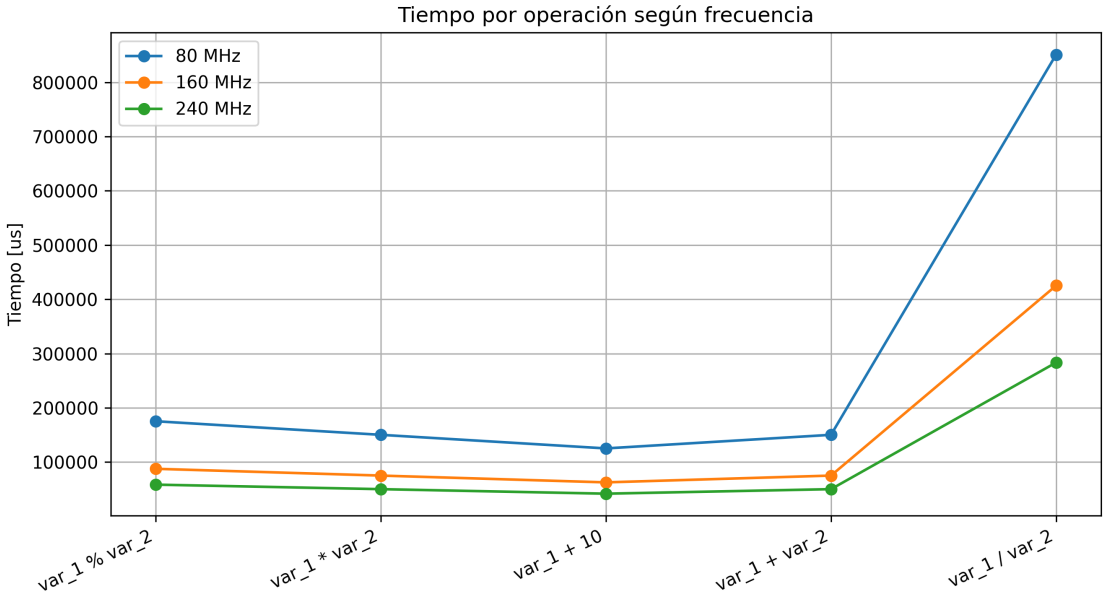


Figura 4: Comparación del tiempo por operación para 80, 160 y 240 MHz.

1.1.3. Impacto de la frecuencia del MCU

El cambio de frecuencia del MCU tiene un impacto muy claro sobre el tiempo total de ejecución del kernel. Para $X = 1000000$, el tiempo medido fue aproximadamente:

- 1401947 μs a 80 MHz
- 700485 μs a 160 MHz

- 466899 μs a 240 MHz

En la práctica, se ve que al aumentar la frecuencia el tiempo baja casi de forma inversamente proporcional. Es decir, al pasar de 80 a 160 MHz el tiempo se reduce casi a la mitad, y al subir a 240 MHz disminuye todavía más.

En cambio, la cantidad total de ciclos se mantiene prácticamente igual para una misma carga de trabajo. Para $X = 1000000$, se midieron aproximadamente 112.16 millones de ciclos a 80 MHz, 112.08 millones a 160 MHz y 112.06 millones a 240 MHz. Esto era esperable, porque el kernel sigue haciendo exactamente el mismo trabajo en todos los casos; lo único que cambia es qué tan rápido se ejecutan esos ciclos.

En general, al aumentar la frecuencia de un microcontrolador, también aumenta la **potencia dinámica**, porque el sistema conmuta más veces por unidad de tiempo. Por eso, trabajar a 240 MHz debería implicar una potencia instantánea mayor que trabajar a 160 MHz o a 80 MHz.

Sin embargo, como el tiempo de ejecución baja al subir la frecuencia, la **energía total** no necesariamente aumenta en la misma proporción. En otras palabras, hay un compromiso entre ejecutar más rápido y consumir más potencia de forma instantánea.

En este experimento no medios potencia directamente. Aun así, los resultados permiten concluir con bastante claridad que la frecuencia afecta fuertemente el tiempo de ejecución y la potencia instantánea, pero no cambia de forma importante la cantidad total de ciclos que necesita el programa.

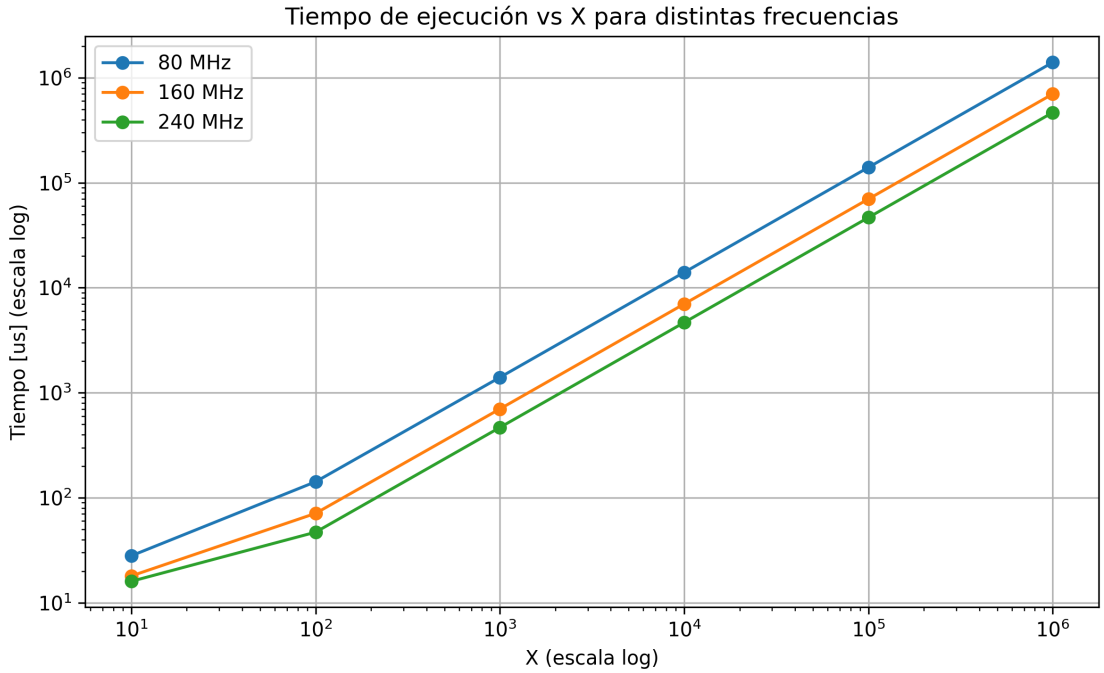


Figura 5: Tiempo de ejecución en función de X para distintas frecuencias del MCU.

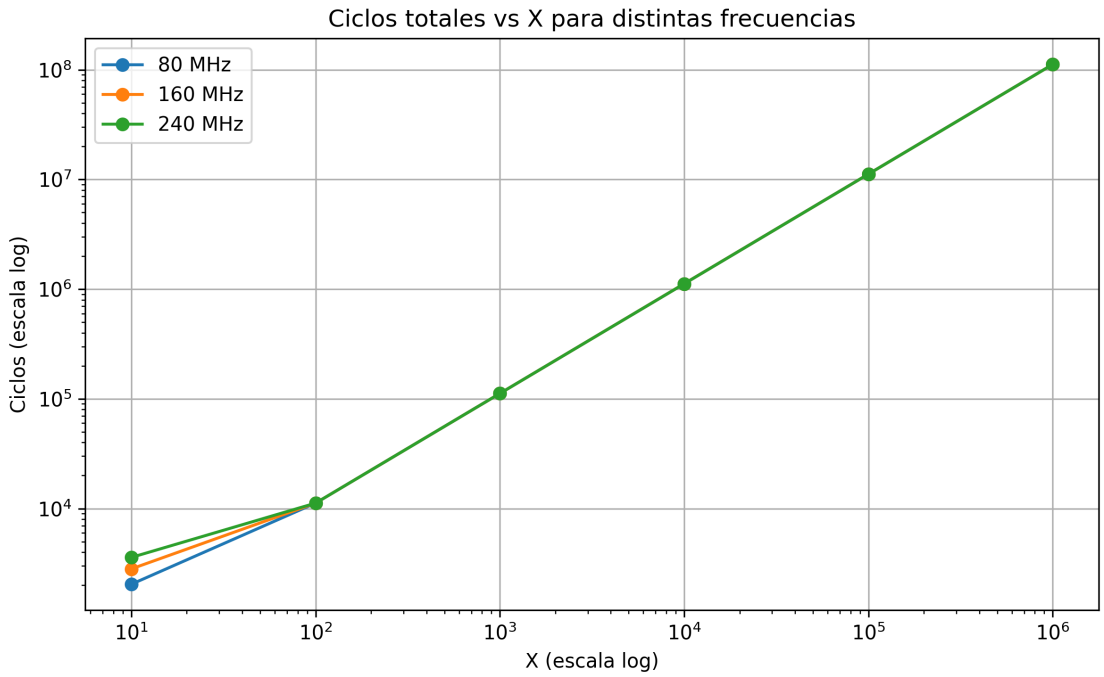


Figura 6: Ciclos totales en función de X para distintas frecuencias del MCU.

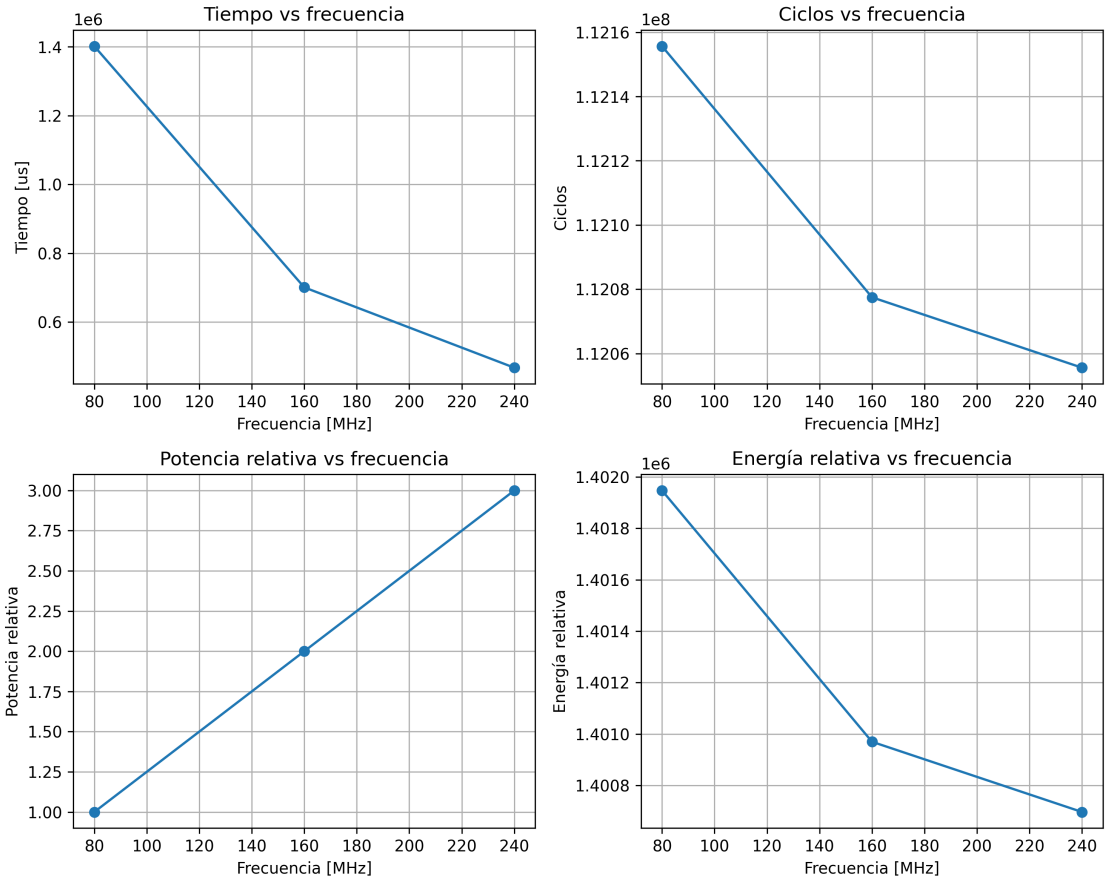


Figura 7: Impacto de la frecuencia sobre tiempo, ciclos, potencia relativa y energía relativa.

1.2. Conclusión

El benchmark implementado permitió entender mejor el comportamiento temporal del kernel propuesto en el Ejercicio 3. Primero, se observó que el CPI estimado converge rápidamente hacia un valor cercano a 6.22–6.23 cuando X aumenta, lo que muestra que el costo por iteración se vuelve estable una vez que el overhead inicial deja de influir tanto. Los valores altos de CPI para X pequeños no reflejan realmente el comportamiento normal del kernel, sino más bien el peso de los costos fijos de medición.

En segundo lugar, el profiling mostró que la división en punto flotante es, por lejos, la operación más costosa del programa y el principal bottleneck. Esta instrucción consume muchos más ciclos que las demás operaciones aritméticas, por lo que termina dominando el tiempo total de ejecución.

Por último, el análisis de frecuencia mostró que al aumentar los MHz del MCU disminuye fuertemente el tiempo de ejecución, mientras que la cantidad total de ciclos se mantiene prácticamente constante. Esto confirma que la frecuencia modifica principalmente la duración temporal de cada ciclo, pero no el trabajo computacional total que realiza el kernel. Desde el punto de vista energético, una mayor frecuencia debería implicar más potencia instantánea, aunque no necesariamente un aumento proporcional en la energía total, justamente porque el tiempo de ejecución se reduce.

En conjunto, los resultados muestran de manera bastante clara cómo métricas como el CPI, el profiling por operación y la frecuencia de reloj permiten entender mejor el desempeño de una aplicación embebida sobre ESP32-S3.

1.3. Anexo

El procesamiento de datos y la generación de gráficos se realizó en Python.

Código utilizado para procesar los datos y generar gráficos:
`https://colab.research.google.com/`